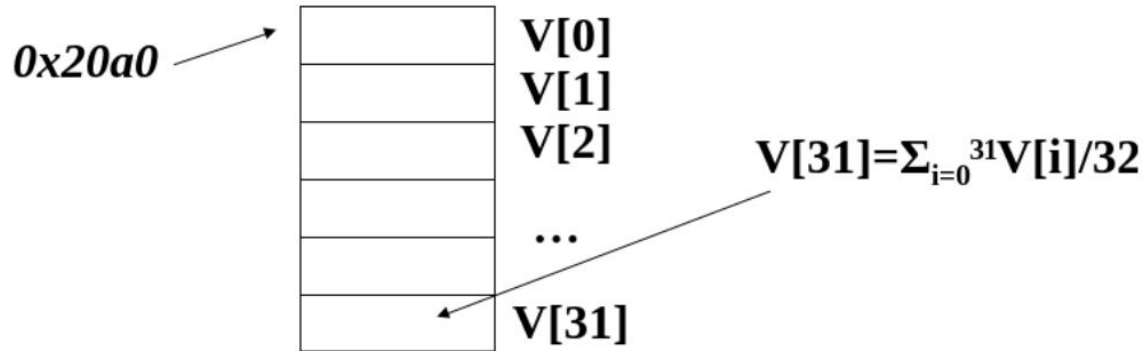


Esercitazione su Assembly

Alessandro Pellegrini
a.pellegrini@ing.uniroma2.it

Calcolo della media di un vettore

- Dato un vettore V di 32 byte senza segno, memorizzato a partire dalla locazione $0x20a0$, calcolarne la media e memorizzarne il valore sovrascrivendo l'ultima posizione del vettore. Se viene rilevato un overflow, in $V[31]$ è posizionato il valore $0xff$ (-1)



Calcolo della media di un vettore

```
.org 0x800
.equ array, 0x20a0 # indirizzo base vettore
.equ dim, 32      # num elementi array
.equ log2dim, 5   # log base 2 di dim
.text
main:
    xorq %rax, %rax # L'accumulatore contiene il risultato parziale
    xorq %rcx, %rcx # Il contatore viene usato come indice
.loop:
    addb array(, %rcx, 1), %al # somma i-esimo elem. i=0..DIM-1
    jc .error                 # se CF = 1 => Overflow
    addq $1, %rcx
    cmpq $dim, %rcx
    jnz .loop                 # continua se non siamo alla fine
```

Calcolo della media di un vettore

```
    shrb $log2dim, %al # dividi per il numero di elementi
    jmp .scrivi
.error:
    movb $0xFF, %al # Sovrascrivi la media con -1
    movq $dim, %rcx
.scrivi:
    subq $1, %rcx
    movb %al, array(, %rcx, 1)
    hlt
```

Ordinamento di un vettore

```
.org 0x800
```

```
.data
```

```
    .comm    array, 4096 # vettore di byte
```

```
    .comm    dim, 4 # intero per dimensione
```

```
.text
```

```
main:
```

```
    xorq %rbx, %rbx # %rbx contiene l'indice del minimo
```

```
.ciclo_esterno:
```

```
    leaq 1(%rbx), %rcx # contiene l'idx dell'elemento da confrontare
```

```
    movq %rbx, %rdx # il minimo si trova all'indice del minimo
```

Ordinamento di un vettore

.ciclo_interno:

```
movb array(, %rdx, 1), %al # carica il minimo corrente
cmpb array(, %rcx, 1), %al # confronta con l'altro elemento
js .skip # se array[%rdx] < array[%rcx]
jz .skip
movq %rcx, %rdx # ho trovato un nuovo minimo, salvo l'indice
```

.skip:

```
addq $1, %rcx # passa all'elemento di confronto successivo
cmpq dim, %rcx # continua se non sono alla fine
jnz .ciclo_interno
```

Ordinamento di un vettore

.ciclo_interno:

movb array(, %rdx, 1), %al # carica il minimo corrente

cmpb array(, %rcx, 1), %al # confronta con l'altro elemento

È sempre
corretto?

js .skip # se array[%rdx] < array[%rcx]

jz .skip

movq %rcx, %rdx # ho trovato un nuovo minimo, salvo l'indice

.skip:

addq \$1, %rcx # passa all'elemento di confronto successivo

cmpq dim, %rcx # continua se non sono alla fine

jnz .ciclo_interno

Ordinamento di un vettore

```
cmpq %rbx, %rdx # verifica se ho trovato un nuovo minimo
jz .dontswap
movb array(, %rbx, 1), %r8b # scambia i valori
movb array(, %rdx, 1), %r9b
movb %r9b, array(, %rbx, 1)
movb %r8b, array(, %rdx, 1)
```

.dontswap:

```
addq $1, %rbx # l'indice del minimo viene incrementato
cmpq dim, %rbx # ripeti se non sono alla fine
jnz .ciclo_esterno
hlt
```


Moltiplicazione tra numeri positivi

- L'assembly z64 include le istruzioni `imul` e `idiv`, ma non abbiamo progettato alcuna componente hardware per eseguire moltiplicazioni e divisioni
- Vanno quindi considerate come pseudo-operazioni che devono essere sostituite da frammenti di codice appropriati
- La soluzione di base naif per la moltiplicazione si basa sull'uguaglianza:

$$a \cdot b = \sum_{i=0}^{b-1} a$$

Moltiplicazione: soluzione naif

```
.org 0x800
```

```
.data
```

```
op1: .long 6
```

```
op2: .long 3
```

```
.text
```

```
xorl %eax, %eax # Accumulatore del risultato
```

```
movl op2, %ecx # Registro utilizzato per la terminazione
```

```
.ciclo:
```

```
addl op1, %eax # Un gran numero di accessi in memoria!
```

```
jc .overflow # Se incorriamo in overflow...
```

```
subl $1, %ecx
```

```
cmpl $0, %ecx
```

Moltiplicazione: soluzione naif

```
jz .fine  
jmp .ciclo
```

```
.overflow:
```

```
movl $-1, %eax # ...impostiamo il risultato a -1
```

```
fine:
```

```
hlt
```

Esercizio per casa: imul e idiv

- `imul` opera su interi segnati
- Aniché scrivere -1 nel risultato in caso di overflow, scrive il risultato in due registri (RDX:RAX), usando RDX per la parte alta
- `idiv` opera su interi segnati
- Restituisce in RAX il quoziente, in RDX il resto
- Può essere implementata con sottrazioni successive
- Usando lo schema della moltiplicazione naif, provate a implementare il codice equivalente delle due istruzioni

Moltiplicazione tra numeri positivi: shift-and-add

- È più efficiente della soluzione naif vista prima
- Ha un costo logaritmico nella dimensione del moltiplicando
- Si basa sull'iterazione di operazioni di shift e somma:
 - Imposta il risultato a 0
 - Ciclo:
 - Shift del moltiplicatore verso sinistra, fino ad allinearlo all'1 meno significativo del moltiplicando
 - Somma il moltiplicatore shiftato al risultato
 - Imposta a 0 il bit meno significativo del moltiplicando impostato a 1
 - Ripeti finché il moltiplicando è uguale a 0

Shift and add: un esempio

- Eseguiamo la moltiplicazione tra 22 e 5:

$$\begin{array}{r} 10110 \quad \times \\ 101 \\ \hline 0 \end{array}$$

Shift and add: un esempio

- Eseguiamo la moltiplicazione tra 22 e 5:

The diagram illustrates the shift and add multiplication process for the binary numbers 10110 (22) and 101 (5). It shows two vertical multiplication problems side-by-side. The first problem shows the initial setup: 10110 multiplied by 101, with a horizontal line under the multiplier 101 and a 0 written below it. The second problem shows the result of a left shift: the multiplier is now 1010 (with the last 0 in red), and the multiplicand 10110 has been shifted one position to the left, resulting in 101100 (with the last 0 in red). A red arrow points from the 0 in the first problem to the red 0 in the second problem, indicating the shift. A blue curved arrow points from the red 0 in the second problem down to the next row, indicating the next step in the process. A plus sign (+) is placed to the right of the second problem, indicating that the results are to be added.

$$\begin{array}{r} 10110 \times 101 \\ \hline 0 \end{array} \quad \begin{array}{r} 10110 \\ 1010 \\ \hline 101100 \end{array} +$$

Shift and add: un esempio

- Eseguiamo la moltiplicazione tra 22 e 5:

The diagram illustrates the shift and add multiplication process for 22 (10110) and 5 (101). It shows three stages of the calculation:

- Initial Setup:** 10110 × 101. A horizontal line is under the 101. A red arrow points from the 0 in the 101 to the 0 in the 10110.
- First Shift:** The 10110 is shifted left by one position, resulting in 101100. A blue curved arrow indicates the shift from the original 10110 to the new 101100.
- Second Shift:** The 10110 is shifted left by two positions, resulting in 1011000. A blue curved arrow indicates the shift from the original 10110 to the new 1011000.

The final result is 11110, which is the sum of 101100 and 1011000.

10110	×	10110		10100	
101		1010		10100	
0		0	+	1010	+
		1010		10100	
		1010		11110	

Shift and add: un esempio

- Eseguiamo la moltiplicazione tra 22 e 5:

The diagram illustrates the shift and add multiplication process for 22 (10110) and 5 (101). It shows four stages of the calculation, with red arrows indicating the shifting of partial products and blue arrows indicating the addition of shifted values.

10110	×	10110		10100		10000
101		1010		10100		1010000
0		0	+	1010	+	11110
		1010		10100		1010000
		1010		11110		1101110

Red arrows show the shifting of the multiplier 101 to the left in each stage. Blue arrows show the addition of the shifted multiplier to the running total.

Shift and add: un esempio

- Eseguiamo la moltiplicazione tra 22 e 5:

The diagram illustrates the shift and add multiplication process for 22 (10110) and 5 (101). It shows four stages of the process:

- Initial Setup:** 10110 × 101. The multiplier 101 has a 1 in the least significant bit, and the product is 0.
- First Shift:** The multiplier bit is 1, so the multiplicand 10110 is shifted left by one position (indicated by a blue arrow) and added to the product. The new product is 1010.
- Second Shift:** The multiplier bit is 0, so the multiplicand is shifted left by one position (indicated by a blue arrow) and added to the product. The new product is 10100.
- Third Shift:** The multiplier bit is 1, so the multiplicand is shifted left by one position (indicated by a blue arrow) and added to the product. The new product is 11110.
- Fourth Shift:** The multiplier bit is 0, so the multiplicand is shifted left by one position (indicated by a blue arrow) and added to the product. The new product is 1101110.

Red arrows indicate the shifting of the multiplicand to the left by one position in each step. Blue arrows indicate the addition of the shifted multiplicand to the product.

Il risultato della moltiplicazione è 1101110 (110)

Adattiamo il codice

- Ogni algoritmo, per essere efficiente, deve essere adattato all'architettura sottostante
- Questo è parte del lavoro che compie il compilatore
- Lo z64 non ha un'istruzione per calcolare il *least significant bit set* (cioè la posizione del bit meno significativo impostato a 1)
- L'x86 (CISC) ha l'istruzione `bsf` (Bit Scan Forward) per questo scopo
- Sarebbe necessario utilizzare un ciclo con all'interno uno shift e un contatore
 - costo troppo elevato

Shift and add adattato

- Possiamo utilizzare un doppio shift, sul moltiplicando e sul moltiplicatore:
 - Shift a destra del moltiplicando: se $CF = 1$, devo sommare il moltiplicatore
 - Shift a sinistra del moltiplicatore
 - Ripeto il ciclo

Moltiplicando:	10110	CF = ?
Moltiplicatore:	101	
Risultato:		0

Shift and add adattato

- Possiamo utilizzare un doppio shift, sul moltiplicando e sul moltiplicatore:
 - Shift a destra del moltiplicando: se $CF = 1$, devo sommare il moltiplicatore
 - Shift a sinistra del moltiplicatore
 - Ripeto il ciclo

Moltiplicando:	1011	$CF = 0$
Moltiplicatore:	101	
Risultato:		0

Shift and add adattato

- Possiamo utilizzare un doppio shift, sul moltiplicando e sul moltiplicatore:
 - Shift a destra del moltiplicando: se $CF = 1$, devo sommare il moltiplicatore
 - Shift a sinistra del moltiplicatore
 - Ripeto il ciclo

Moltiplicando:	1011	CF = 0
Moltiplicatore:	1010	
Risultato:		0

Shift and add adattato

- Possiamo utilizzare un doppio shift, sul moltiplicando e sul moltiplicatore:
 - Shift a destra del moltiplicando: se $CF = 1$, devo sommare il moltiplicatore
 - Shift a sinistra del moltiplicatore
 - Ripeto il ciclo

Moltiplicando:	101	$CF = 1$
Moltiplicatore:	1010	
Risultato:	1010	

Shift and add adattato

- Possiamo utilizzare un doppio shift, sul moltiplicando e sul moltiplicatore:
 - Shift a destra del moltiplicando: se $CF = 1$, devo sommare il moltiplicatore
 - Shift a sinistra del moltiplicatore
 - Ripeto il ciclo

Moltiplicando:	101	$CF = 1$
Moltiplicatore:	10100	
Risultato:	1010	

Shift and add adattato

- Possiamo utilizzare un doppio shift, sul moltiplicando e sul moltiplicatore:
 - Shift a destra del moltiplicando: se $CF = 1$, devo sommare il moltiplicatore
 - Shift a sinistra del moltiplicatore
 - Ripeto il ciclo

Moltiplicando:	10	$CF = 1$
Moltiplicatore:	10100	
Risultato:	11110	

Shift and add adattato

- Possiamo utilizzare un doppio shift, sul moltiplicando e sul moltiplicatore:
 - Shift a destra del moltiplicando: se $CF = 1$, devo sommare il moltiplicatore
 - Shift a sinistra del moltiplicatore
 - Ripeto il ciclo

Moltiplicando:	10	$CF = 1$
Moltiplicatore:	101000	
Risultato:	11110	

Shift and add adattato

- Possiamo utilizzare un doppio shift, sul moltiplicando e sul moltiplicatore:
 - Shift a destra del moltiplicando: se $CF = 1$, devo sommare il moltiplicatore
 - Shift a sinistra del moltiplicatore
 - Ripeto il ciclo

Moltiplicando:	1	$CF = 0$
Moltiplicatore:	101000	
Risultato:	11110	

Shift and add adattato

- Possiamo utilizzare un doppio shift, sul moltiplicando e sul moltiplicatore:
 - Shift a destra del moltiplicando: se $CF = 1$, devo sommare il moltiplicatore
 - Shift a sinistra del moltiplicatore
 - Ripeto il ciclo

Moltiplicando:	1	$CF = 0$
Moltiplicatore:	1010000	
Risultato:	11110	

Shift and add adattato

- Possiamo utilizzare un doppio shift, sul moltiplicando e sul moltiplicatore:
 - Shift a destra del moltiplicando: se $CF = 1$, devo sommare il moltiplicatore
 - Shift a sinistra del moltiplicatore
 - Ripeto il ciclo

Moltiplicando:	0	$CF = 1$
Moltiplicatore:	1010000	
Risultato:	11110	

Shift and add adattato

- Possiamo utilizzare un doppio shift, sul moltiplicando e sul moltiplicatore:
 - Shift a destra del moltiplicando: se $CF = 1$, devo sommare il moltiplicatore
 - Shift a sinistra del moltiplicatore
 - Ripeto il ciclo

Moltiplicando:	0	$CF = 1$
Moltiplicatore:	1010000	
Risultato:	<u>1101110</u>	

Shift and add: il codice

```
.org 0x800
```

```
.data
```

```
op1: .long 3
```

```
op2: .long 2
```

```
.text
```

```
movl op1, %ebx # carico il moltiplicando
```

```
movl op2, %edx # carico il moltiplicatore
```

```
xorl %eax, %eax # %eax conterrà il risultato
```

```
cmpl $0, %edx
```

```
jz .fine # Non devo moltiplicare per zero
```

Shift and add: il codice

`.moltiplica:`

`cmpl $0, %ebx`

`jz .fine` # Quando il moltiplicando è zero ho finito

`shrl $1, %ebx`

`jnc .nosomma` # Se C = 0 non sommo

`addl %edx, %eax`

`.nosomma:`

`shll $1, %edx`

`jc .overflow` # Controllo l'overflow

`jmp .moltiplica`

Shift and add: il codice

.overflow:

```
movl $-1, %eax # In caso di overflow, imposto il risultato a -1
```

.fine:

```
hlt
```

Calcolo del fattoriale

```
.org 0x800
.data
    numero: .quad 20
    risultato: .fill 1, 8
.text
    movq numero, %rdi
    call fattoriale
    testq %rdx, %rdx    # Verifica la presenza di overflow
    jz .corretto
    movq $-1, risultato
    hlt
.corretto:
    movq %rax, risultato
    hlt
```

Calcolo del fattoriale

```
# Subroutine per la moltiplicazione
# Parametri in %rdi e %rsi
# Valore di ritorno in %rax
# Se avviene overflow, ritorna con %rdx=1
```

moltiplica:

```
    xorq %rax, %rax
    xorq %rdx, %rdx
    testq %rdi, %rdi
    jz .done_mult
.loop_mult:
    addq %rsi, %rax
    jc .overflow_mult
    subq $1, %rdi
    jnz .loop
    ret

.overflow_mult:
    movb $1, %rdx
.done_mult:
    ret
```

Calcolo del fattoriale

Subroutine per il fattoriale

Parametro in %edi

Valore di ritorno in %eax

Se avviene overflow, ritorna con RDX=1

fattoriale:

```
xorq %rax, %rax
```

```
push %rdi
```

```
cmpl $1, %rdi      # Se %rdi = 1, esegui il passo base
```

```
jnz .passoricorsivo
```

```
movl %rdi, %rsi
```

```
jmp .passobase
```

Calcolo del fattoriale

.passoricorsivo:

subq \$1, %rdi

call fattoriale # Chiama la stessa subroutine

testq %rdx, %rdx

jz .nooverflow

pop %rdi # Con overflow, tolgo dallo stack i valori salvati

jmp .doneFATT

.nooverflow:

movq %rax, %rsi

.passobase:

pop %rdi

call moltiplica # %rax = n * fatt(n-1)

.doneFATT:

ret